Proving Ethereum with Lattices (Extended Abstract)

Lukáš Častven
Slovak University of Technology in Bratislava

Abstract. Ethereum scalability is bottlenecked by the need to repeatedly execute EVM computation on many nodes. Snarkifying execution would replace much of that work with verification of a compact proof. Most current ZkVMs obtain post-quantum security from hash based STARKs, while some recursive systems still rely on elliptic curve assumptions. This work studies a third path: a general purpose RISC-V ZkVM built from lattice cryptography. The design combines CCS arithmetization, LatticeFold for incrementally verifiable computation, deferred memory checking, and a Greyhound wrapper proof. The evaluation is based on an incomplete Rust implementation. The prototype is much slower than mature ZkVMs, but the design is feasible, longer runs keep memory bounded, and the final proof in the design is expected to be far smaller than current ZkVM proofs, in the tens of kilobytes rather than hundreds of kilobytes to a megabyte.

Motivation

Ethereum's long-term scalability depends on replacing widespread reexecution with succinct proofs of correct computation [1]. For execution, this appears either as zkEVMs that model the EVM directly or as ZkVMs that prove an easier virtual machine and embed Ethereum logic inside it [2]. In practice, many systems have converged on RISC-V ZkVMs because the EVM's design (stack oriented, use of Keccak256, ...) is difficult to arithmetize efficiently.

At the proving layer, the design space is split between hash-based STARKs and recursive SNARKs based on elliptic curves. STARKs provide post-quantum security but often with larger proofs or heavier verification [3]. Curve recursion can be compact, but its assumptions are not quantum resistant. Lattice based cryptography offers a third option: post-quantum security from algebraic hardness assumptions together with commitments that support folding based recursion [4, 5, 6]. Despite recent progress, there is still no practical, general purpose RISC-V ZkVM built from these ingredients. The work addresses that gap and asks whether lattice based folding can support the long computations required for Ethereum block validation.

Architecture

The proposed system follows the layered structure common in modern ZkVMs [2]. A guest program is compiled to a 32 bit RISC-V binary and executed instruction by instruction by an emulator. Each step emits a trace containing the state of the VM before and after, such as program counter, register state, decoded instruction, and memory side effects. This trace is arithmetized into a witness over Goldilocks ring elements.

The core relation is expressed as a Customizable Constraint System (CCS) [7]. A step circuit F enforces correct RISC-V transitions, including decoding, operand validation, arithmetic or control flow semantics, and committed memory page updates. To make long executions provable with folding, the F is wrapped with an augmented circuit F' that instantiates incrementally verifiable computation (IVC) [8, 9, 10]. For each instruction, F' checks a commitment to the machine state at the end of last step, verifies the previous folding proof, and enforces the current RISC-V step. Off circuit, the prover uses LatticeFold to compress the current step and the running accumulator into a new accumulator [6]. This replaces full recursive proof verification at every step with a cheaper folding operation under lattice assumptions.

Memory is handled with deferred checking rather than a full in circuit RAM argument. Every load and store is appended to a transcript during execution, and the final proof verifies permutation, ordering, and read over write properties using offline memory checking techniques [11]. To produce a succinct artifact for an external verifier, the final folded instance is wrapped with a SuperSpartan style IOP combined with Greyhound polynomial commitments [12, 13]. Thanks to Greyhound, final proofs sizes are in the tens of kilobytes, much smaller than the hundreds of kilobytes to about one megabyte reported for current ZkVMs.

Implementation and evaluation

An incomplete Rust prototype is presented for the proving pipeline. The implementation is sufficient for initial experiments. The first benchmark uses a small program that computes the 100th Fibonacci number. This benchmark was evaluated on the prototype implementation together with Pico[1], SP1[2], and ZKsync Airbender[3] on the same machine. In the prototype implementation, the benchmark corresponds to 16 executed instructions and 2 memory operations. The benchmark summary is shown in Table 1.

ZkVM	Proof time	Peak RAM
Prototype implementation	11:28.92	12.34 GB
Pico	2:04.50	6.27 GB
SP1	1:50.17	9.96 GB
ZkSync Airbender	4:06.73	24.34 GB

Table 1. Fibonacci benchmark comparison.

[1] https://pico-docs.brevis.network/.
[2] https://docs.succinct.xyz/docs/sp1/introduction.
[3] https://www.zksync.io/airbender.

The results show that the prototype implementation is slower than Pico, SP1, and ZKsync Airbender on this small benchmark. At the same time, its memory use stays below Airbender and within a manageable range. The gap in proving time is attributed to multilinear extension, decomposition, and folding work in the proving backend rather than to the VM frontend itself.

REVM experiment. The longer `revm` based experiment gives the most encouraging signal for the Ethereum use case. That run continued for more than 9 hours while the IVC chain kept advancing and memory usage remained bounded at roughly 13.7 GB. This does not yet prove readiness for production Ethereum block proving. However, it does show that the design can sustain a much longer recursive execution without obvious runaway memory growth or a breakdown of the folding process. In that sense, the experiment is an encouraging sign that witness norm management and recursive state growth are not fundamental blockers for the approach.

Scientific questions. Answers to the research questions are cautious but positive. The evaluation asks three things: whether it is feasible to instantiate a full RISC-V execution environment using CCS compatible with LatticeFold, how a lattice based folding scheme compares in performance with established ZkVMs, and whether noise growth in lattice cryptography hinders the ability to prove long traces such as Ethereum block validation. The first question is answered partially yes, because the main building blocks can be connected into a single proving pipeline even though the implementation remains incomplete. The second question currently receives a negative practical answer, since the prototype is slower than established ZkVMs. Still, the design retains an important upside: unlike most current systems, it targets compact post-quantum proofs in the tens of kilobytes. The third question receives a cautiously positive answer, because the long `revm` run suggests that long executions are viable enough to justify further work, even though production scale Ethereum block proving remains out of reach.

Identified problems. The evaluation indicates that the main cost is not VM execution itself, but multilinear extension, decomposition, and folding work in the proving backend. The implementation is CPU only and does not yet benefit from SIMD or GPU acceleration. CCS is expressive, but it does not appear to be the best model for a VM together with a recursive verifier, especially for cryptographic components and self referential verifier logic. CCS is convenient for some linear instruction semantics, but becomes awkward for parts such as Poseidon2 and the folding verifier, where the constraint structure starts to work against efficiency. Taken together, these observations suggest that the current bottlenecks are mostly engineering and arithmetization issues rather than evidence that lattice based folding itself is unsuitable.

Future work. The most direct next steps are to replace LatticeFold with improved backends such as LatticeFold+ or Neo, revisit CCS as the computation model for recursion and VM constraints, and exploit SIMD or GPU acceleration for the regular linear algebra workload [14, 15]. Beyond that, further work is motivated on better algebraic layouts for multilinear extensions, reuse of intermediate values across folding steps, and a proving architecture that exposes more parallelism. The broader conclusion is therefore not that the approach is already practical, but that it is concrete enough to optimize and compare as a serious post- quantum direction for Ethereum execution proving.

Conclusion. The work presents a feasible path toward Ethereum proving with lattices. It combines a RISC-V execution model, CCS, folding, and Greyhound into a design that promises compact proofs, while the current prototype shows where the remaining performance gaps lie. The main takeaway is simple: the approach is not ready yet, but it is concrete, measurable, and worth further optimization.

References

[1] V. Buterin. *Ethereum: A Next-Generation Smart Contract and Decentralized Application Platform.* 2014.

[2] Y. Yang, Y. Cheng, H. Tang, G. Yang, B. Zhang, and K. Ren. *SoK: Understanding zkVM: From Research to Practice.* Cryptology ePrint Archive, Paper 2026/525, 2026.

[3] E. Ben-Sasson, I. Bentov, Y. Horesh, and M. Riabzev. *Scalable, Transparent, and Post-Quantum Secure Computational Integrity.* Cryptology ePrint Archive, Paper 2018/046, 2018.

[4] M. Ajtai. *Generating Hard Instances of Lattice Problems (Extended Abstract).* In *Proceedings of STOC*, pages 99–108, 1996.

[5] V. Lyubashevsky, N. K. Nguyen, and M. Plançon. *Lattice-Based Zero-Knowledge Proofs and Applications: Shorter, Simpler, and More General.* Cryptology ePrint Archive, Paper 2022/284, 2022.

[6] D. Boneh and B. Chen. *LatticeFold: A Lattice-based Folding Scheme and its Applications to Succinct Proof Systems.* Cryptology ePrint Archive, Paper 2024/257, 2024.

[7] S. Setty, J. Thaler, and R. Wahby. *Customizable Constraint Systems for Succinct Arguments.* Cryptology ePrint Archive, Paper 2023/552, 2023.

[8] P. Valiant. *Incrementally Verifiable Computation or Proofs of Knowledge Imply Time/Space Efficiency.* In *Theory of Cryptography*, pages 1–18, 2008.

[9] A. Kothapalli, S. Setty, and I. Tzialla. *Nova: Recursive Zero-Knowledge Arguments from Folding Schemes.* Cryptology ePrint Archive, Paper 2021/370, 2021.

[10] A. Kothapalli and S. Setty. *HyperNova: Recursive Arguments for Customizable Constraint Systems.* Cryptology ePrint Archive, Paper 2023/573, 2023.

[11] A. Ozdemir, E. Laufer, and D. Boneh. *Volatile and Persistent Memory for zkSNARKs via Algebraic Interactive Proofs.* Cryptology ePrint Archive, Paper 2024/979, 2024.

[12] S. Setty. *Spartan: Efficient and General-Purpose zkSNARKs without Trusted Setup.* Cryptology ePrint Archive, Paper 2019/550, 2019.

[13] N. K. Nguyen and G. Seiler. *Greyhound: Fast Polynomial Commitments from Lattices.* Cryptology ePrint Archive, Paper 2024/1293, 2024.

[14] D. Boneh and B. Chen. *LatticeFold+: Faster, Simpler, Shorter Lattice-Based Folding for Succinct Proof Systems.* Cryptology ePrint Archive, Paper 2025/247, 2025.

[15] W. Nguyen and S. Setty. *Neo: Lattice-Based Folding Scheme for CCS over Small Fields and Pay-Per-Bit Commitments.* Cryptology ePrint Archive, Paper 2025/294, 2025.